

FlowZ 使用说明

给普通开发者看的说明

1. 一句话说明怎么用

把 FlowZ 文件夹交给 Cline，然后对 Agent 说：

安装 FlowZ Cline 全局工作流，并在当前工作区启用它。

Agent 会先识别当前是 Windows 还是 Ubuntu/Linux，再读取当前 Cline（或兼容版本）的实际配置，自动选择安装命令和安装目录。它会自己读取 FlowZ 里的安装资源，安装全局规则和配套 Skill。找不到唯一的安装目录时，Agent 应先告诉你，不会随便创建一个目录。安装完成后，其他新项目 and 旧项目也可以使用这套工作流，不需要把 FlowZ 文件夹复制到每个项目里。

2. FlowZ 是什么，能干嘛

FlowZ 是一套给 AI 编程 Agent 使用的工作方法。它不能替你写需求，也不是一个自动完成所有事情的插件。

它主要帮助 Agent：

- 先判断任务大小，再选择 Quick、Standard 或 Full；
- 目标不清楚时先把问题问清楚；
- 已有方案时主动找漏洞、反例和证据缺口；
- 需要工程设计时先形成可检查的设计，再开始修改；
- 尽量只读取相关内容，减少重复沟通；
- 修改后说明做了什么、验证了什么，还有什么没有验证；
- 对简单、可观察的结果，告诉你可以怎样自己确认。

安装后，Agent 会在每个工作区的首个任务中读取现有规则和项目状态，尽量接着项目原来的方式工作，不会因为安装了 FlowZ 就强行改动项目结构。

FlowZ 还包含少量通用 Skill，例如中文或英文文本整理、问题探索和方案压力测试。它们只在确实适合时使用，不会每个任务都全部运行。

聊天式AI辅助

FlowZ 会尽量减少重复读取、重复提问和无效输出。对于较长文字的解释、整理、比较或审查，如果聊天式AI只需要你提供文字或一个文本文件就能完成，而且不需要读取项目、修改文件或运行命令，只有在你明确打开辅助后，Agent 才会建议把这部分工作交给聊天式AI处理。

这样做是为了减少当前开发任务中的上下文消耗。它不是强制步骤；你也可以继续在当前 Agent 中完成。聊天式AI不会自动获得项目文件或运行权限，除非你主动提供相应内容。

省 token 不只靠聊天式AI。FlowZ 还会尽量选择最小的任务流程，减少重复读取和重复总结，失败后只携带新增证据，并把能由用户直接完成的部分命令和简单验证方法告诉你。你可以按提示自己运行命令、查看结果，再把结果反馈给 Agent，这同样能减少不必要的对话和调用。

聊天式AI辅助默认关闭。需要时直接对 Agent 说：

打开聊天式AI辅助

如果不想继续使用，可以说：

关闭聊天式AI辅助

用户明确要求时，Agent 应按要求执行。没有收到明确的打开或开启指令前，不主动提出这类建议。

3. FlowZ 的优点：我为什么这样设计

这套设计来自一个很实际的麻烦：每次都从头向 AI 解释项目很浪费时间，也很容易前后说得不一致。后来我发现，真正重要的不是把 Prompt 写得多漂亮，而是把 AI 完成任务需要知道的内容说完整。

我不希望 AI 一上来就开始改代码，也不希望每个问题都走一套很重的流程。简单问题应该快一点，复杂问题应该认真一点。

所以 FlowZ 做了几件事：

- 小改动走轻流程，避免为简单任务浪费时间和上下文；
- 复杂任务先把目标、范围、风险和验收条件说清楚；
- 找问题、找漏洞和做工程设计分开处理，避免多个能力互相重复提问；
- 阶段之间传递简短结论，不反复粘贴整段历史；
- 出现失败时只带上新增证据，不从头重复一遍；
- 对聊天式AI可以独立完成的较大文字任务，提供一个可选的辅助通道，减少当前 Agent 的重复上下文；
- 尽量把简单验证方法告诉你，让你能亲自看到结果；
- 关键检查仍然由 Agent 负责，不能因为追求简短就跳过必要验证。

它还把不同类型的信息分开处理：长期习惯、项目背景、当前任务、重复工作的做法和跨会话状态，不再全部混在一段聊天里。不同工具可能把这些东西叫作规则、Skill 或 handoff，但目的都是一样的：让 AI 知道哪些内容长期有效，哪些只对这一次任务有效。

我也更倾向于先把核心功能做出来，再根据真实使用继续调整。先做出能用的版本，比一开始就把所有附加功能都加进去更容易控制方向。真正使用的人遇到的麻烦，往往比开发者坐在电脑前想象出来的更具体。

这样做是为了让 AI 更专注，也让你更容易看出它现在处于什么阶段、准备做什么、依据是什么。

4. FlowZ 的不足

FlowZ 不能解决 AI 本身的问题。

- Agent 仍然可能理解错你的意思，或者漏掉重要条件；
- Agent 可能把一个看似简单的问题判断得太复杂，也可能反过来低估风险；
- Cline 的版本、规则加载方式和 Plan / Act 行为可能变化；
- 打开项目本身不一定会立刻触发工作流，通常是在 Agent 处理这个工作区的首个任务时接入；
- 项目原有规则可能和 FlowZ 不同，遇到冲突时需要以实际项目要求为准；
- 聊天式 AI 返回的结果可能不完整或不准确，不能直接当成项目事实；
- FlowZ 不会替你做最终决定，也不会替你承担代码、数据或发布风险。
- 即使自己已经尽量站在使用者角度思考，真实使用后仍然会发现边角问题和新 Bug；持续反馈和修正仍然不可避免。

如果 Cline 没有自动加载刚安装的规则，可以重新发送安装指令，或者先刷新、重载 Cline。

5. 极为重要：AI 不是许愿机

这是使用 FlowZ 时最重要的一点。

不要把 AI 当成一个只要说出愿望，就会自动给出正确结果的机器。即使安装了 FlowZ，也不能只聊天、不思考，然后把决定全部交给 AI。

你需要主动参与：

- 说清楚你真正想解决的问题，而不只是说“帮我优化一下”；
- 需求不符合预期时，先回头检查自己有没有漏说背景、目标或限制；
- 补充范围、限制、不能改什么、什么结果才算完成；
- 需求还没有想明白时，不要让 AI 替你猜，也不要急着让它开始修改；
- 经常问 AI：你现在理解的目标是什么？你做了哪些假设？还有哪些地方不确定？
- 要求 AI 说明为什么这样做，而不是只接受一个看起来合理的答案；
- 让 AI 说出可能失败的情况、遗漏的条件和没有检查的部分；
- 关注它有没有偷偷扩大范围，或者偏离你原来的方向；
- 观察它准备做什么、依据什么、修改什么、执行什么，发现方向不对就及时打断；
- 多站在第一次使用这个功能的人角度想一遍，不要把自己的熟悉当成用户也应该知道；
- 看实际改动、验证结果和未执行的检查，不要只看它的总结；

- 对重要改动、数据变化、权限变化和发布操作，自己再确认一次；
- 发现它理解错了，就立刻纠正，不要为了让对话继续而默认接受；
- 不要让长时间的讨论代替真正的实现、验证和使用结果。

不要因为 AI 已经“改完了”就默认任务已经完成。功能是否符合目标、操作是否真的正常，最后仍然要由人验收。自动检查能帮你发现一部分问题，但不能替你判断这个结果是不是你真正想要的。

一个好的使用方式不是“我提出愿望，AI 负责变魔法”，而是：你提出目标和边界，AI 帮你调查、提出方案、执行和验证；你始终理解它在做什么，并决定是否继续。

如果你不知道该问什么，可以直接问：

请先告诉我你对目标、范围、限制和验收标准的理解。

列出你做出的假设、可能遗漏的情况，以及你准备如何验证。

如果我的要求不够清楚，请先指出缺口，不要急着修改文件。

FlowZ 能帮你把工作过程整理得更可靠，但真正决定结果质量的，仍然是你是否愿意思考、追问、补充条件，并且亲自判断 AI 有没有走偏。